최기환

Frontend Engineer

rlghks3004@gmail.com GitHub gihwan-dev.tistory.com

현재 통합 모니터링 대시보드와 exem-table 라이브러리, MaxGauge 레거시 5개 제품의 프론트엔드 개발과 유지보수를 혼자 맡고 있습니다. 차세대 MaxGauge의 화면 개발과 공통 컴포넌트 개선도 병행하고 있습니다. 입사부터 만 2년차가 된 지금까지 팀에서 관리하는 모든 제품에 기여했으며, 빠르게 업무 영역을 넓혀가고 있습니다.

01 소프트웨어가 계속해서 나아질 수 있는 환경을 설계합니다. 새 제품과 레거시에 테스트 코드를 도입해, 코드를 고친 뒤에도 기존 기능이 제대로 동작하는지 확인합니다.

02 좋은 개발자는 회사에 돈을 잘 벌어주는 개발자라고 생각합니다. 업무의 단기적 가치와 장기적 가치, 필요한 리소스를 저울질하며 선택하려고 노력합니다.

03 귀찮은 일을 가장 싫어합니다. 반복되는 작업과 불필요한 일을 최대한 줄이려고 합니다.

경력

주식회사엑셈 / 프론트엔드 개발자 2024.11 - 현재

exem-table | 사내 React 테이블 라이브러리 2025.06경 인수 - 현재

`React / TypeScript / TanStack Table / Vitest`

DB 성능 분석 화면과 통합 대시보드에서 사용하는 테이블 라이브러리입니다. 기존 컴포넌트를 인수해 API와 렌더링 구조를 바꾸고, 사용하는 제품의 전환까지 직접 맡았습니다.

- **상태를 누가 관리하는지 분명한 API로 재설계**
  - 화면과 테이블 컴포넌트, TanStack Table이 상태를 중복 관리해 값이 바뀔 때마다 서로 맞춰 주는 코드가 필요했습니다.
  - 정렬과 검색 등 기능별로 옵션을 묶고, 화면이 상태를 관리하는 제어 방식과 라이브러리에 맡기는 비제어 방식으로 API를 정리했습니다.
  - 테이블 컴포넌트가 따로 보관하던 중간 상태를 없애 동기화 코드를 줄였습니다.
- **기능을 두 번 구현하지 않도록 가상화를 기본 동작으로 선택**
  - 기존 table 태그 구조는 컬럼 리사이즈와 이동 UI를 배치하는 데 제약이 있어 div 기반으로 바꿨습니다.
  - 가상화를 선택 옵션으로 두면 같은 기능을 두 경로에서 구현하고 검증해야 했습니다. 이 부담이 가상화의 제약보다 크다고 판단해 기본 동작으로 정했습니다.
  - 컬럼 고정과 행 확장도 같은 가상화 구조에서 처리하도록 만들었습니다.
- **차세대 MaxGauge의 22개 화면에 공통 테이블 적용**
  - 화면마다 달랐던 정렬과 필터 동작은 라이브러리로 모으고, 제품별 컬럼 정의와 상세 화면 이동 규칙은 별도 어댑터에 뒀습니다.
  - 필요한 라이브러리 수정과 22개 화면의 전환을 직접 수행해, 각 화면은 컬럼과 데이터만 지정하도록 바꿨습니다.
  - 259개 컬럼 정의를 한곳에서 관리해 이름과 타입이 바뀔 때 화면마다 수정하던 작업을 없앴습니다.

exem-table (계속)

- **기능을 검증하는 테스트를 바탕으로 v9 전환과 렌더링 비용 개선**
  - 내부 구조를 바꾸자 함께 깨지던 테스트를 공개 API와 사용자 조작 중심으로 다시 작성했습니다. 정렬과 검색, 선택, 리사이즈를 실제 브라우저에서 검증하도록 했습니다.
  - 최상위 컴포넌트가 전체 상태를 구독해 값 하나가 바뀌어도 테이블 전반이 다시 렌더링됐습니다. TanStack Table v9로 이전하면서 각 컴포넌트가 필요한 상태만 구독하도록 바꿨습니다.
  - 10,000행과 8개 컬럼에서 14자 검색어를 입력할 때 누적 React 렌더 시간이 1,927ms에서 61ms로 줄었습니다. 당시 동일 조건에서 5회 측정한 중앙값입니다.
  - 사용하는 제품도 새 API로 옮기면서, 타입 검사는 통과하지만 실제로는 적용되지 않던 커스텀 정렬을 4개 화면에서 수정했습니다.

관련 글: Tanstack Table V9 마이그레이션 회고

MaxGauge 레거시 제품군 | 고객 요청부터 패키징까지 전담 `2026.09부터 전담`

`JavaScript / ExtJS / Playwright / Claude Code / Codex`

DB 성능을 모니터링하는 MFO, MFT, MFA, MFD, MFS의 프론트엔드를 혼자 맡고 있습니다. 고객 요청 분석부터 재현과 구현, 검증, 버전 관리와 패키징까지 담당하고 있습니다.

- **AI가 수정한 결과를 실제 화면으로 검토하는 작업 절차 구축**
  - 화면의 각 영역에 사용할 API와 동작 규칙을 연결하는 스킬을 만들었습니다. 구현 전에 수정 의도와 설계를 화면을 보며 검토할 수 있게 했습니다.
  - 과거 MR에서 반복된 버그와 사이드 이펙트를 조사해 AI 코드 리뷰의 점검 항목으로 만들었습니다.
  - 테스트와 AI 코드 리뷰를 거친 결과를 시나리오별 이미지로 남기도록 했습니다. 매번 코드를 세세히 읽거나 직접 재현하지 않고도 화면에서 의도한 동작을 확인하며 병합할 수 있게 됐습니다.
  - 현재 커밋에 대한 리뷰 기록이 없으면 도구가 MR 작성과 패키지 전달을 막도록 했습니다.
- **레거시 수정에 회귀 테스트 도입**
  - MFO에 테스트 코드를 도입하고, 기능을 수정할 때마다 기존 동작이 깨지지 않는지 확인하고 있습니다.
  - 고객 패치에서는 같은 입력으로 기존 버전의 차트와 알람 표시 오류를 재현하고, 수정 버전에서 테스트가 통과하는지 확인했습니다. 검증 화면도 함께 남겨 변경 전후를 비교할 수 있게 했습니다.
- **버전과 배포 소스를 추적할 수 있도록 전달 절차 정비**
  - 어떤 버전에 어떤 작업이 포함됐는지 추적하기 어려워, 작업 목록과 클라이언트 및 SQL의 소스 커밋을 버전 태그에 연결했습니다.
  - 이미 사용한 버전 번호를 선택하거나 검토 후 배포 대상 소스가 바뀌면 진행을 막도록 했습니다.
  - 실제 고객 패치에도 이 절차를 적용해 소스와 버전 태그를 확정하고, 해당 소스로 패키지를 생성한 기록을 남겼습니다.
  - 공통 개발과 릴리스, 고객사 브랜치의 역할 및 수정 반영 규칙을 다시 정했고, 기존 브랜치의 전환을 진행하고 있습니다.

MaxGauge Dashboard | 통합 인스턴스 모니터링 대시보드

입사 초기부터 담당

`Vue에서 React 및 TypeScript로 전환`

여러 DB 인스턴스의 상태와 알람을 한 화면에서 확인하는 제품입니다. 별도 기획자와 디자이너 없이 고객 요구사항을 받아 기획과 디자인, 개발을 맡았습니다. 기존 Vue Dashboard 6.0을 React로 전환했으며 현재 고객사에서 운영 중입니다.

- **고객사별 브랜치를 기능 설정으로 통합**
  - 고객사별 브랜치에서 기능을 따로 개발해, 공통 버그를 고칠 때마다 각 브랜치에 수정 내용을 옮겨야 했습니다.
  - 모든 고객사에 제공하는 공통 제품으로 방향을 정하고, 필요한 기능은 고객사별 설정에서 켜거나 끄도록 바꿨습니다.
  - 같은 코드 기반으로 고객사별 기능을 제공하면서 공통 수정을 여러 브랜치에 전파하던 작업을 없앴습니다.
- **사내 React 디자인 시스템을 활용해 화면 개발 방식 통일**
  - 혼자 기획과 디자인을 맡다 보니 작업마다 화면 스타일이 달라졌습니다.
  - 사내 디자인 시스템을 활용할 수 있도록 React로 이전하고, 공통 컴포넌트와 테마 시스템을 적용했습니다.
  - 공통 컴포넌트와 테마를 재사용해 화면의 디자인을 일관되게 맞추고, 기능 개발에 드는 작업도 줄였습니다.
- **TypeScript와 테스트를 도입해 후속 변경 검증**
  - TypeScript 기반으로 전환하고, 사용자 조작에 따른 화면 결과를 확인하는 테스트를 추가했습니다.
  - 화면 구조나 데이터 처리 코드를 수정한 뒤에도 기존 기능이 유지되는지 테스트로 확인하고 있습니다.

MaxGauge 차세대 | DB 성능 분석 제품 개발

`React / TypeScript / TanStack Query / ECharts`

SQL과 세션, DB 객체를 분석하는 제품입니다. 초기에는 위젯 빌더와 SQL Detail, Object Detail을 개발했고, 현재는 제품 전반의 기능 수정과 공통 컴포넌트 개선에 참여하고 있습니다.

- **새 시각화를 플러그인으로 추가할 수 있는 위젯 빌더 개발**
  - 시각화 종류별로 흩어져 있던 설정 폼과 검증 규칙, 미리보기를 하나의 플러그인에 모았습니다.
  - 새 차트나 테이블 미리보기를 추가할 때 해당 플러그인 안에서 관련 코드를 함께 작성하고 수정할 수 있도록 했습니다.
- **차트에서 선택한 SQL을 먼저 보여주도록 조회 방식 변경**
  - 선택한 SQL이 목록 뒤쪽에 있으면 앞 페이지를 모두 받아야 했던 조회를, 해당 SQL이 포함된 페이지부터 받도록 바꿨습니다.
  - 앞뒤 데이터는 스크롤할 때 조회하고 아직 받지 않은 영역만 로딩으로 표시해, 추가 조회 중에도 기존 행을 선택하고 스크롤할 수 있게 했습니다.
- **일반적인 사용 경험을 해치던 차트 리사이즈 최적화 제거**
  - 캔버스를 늘려 보여주고 200ms 뒤 다시 그리는 최적화 때문에 리사이즈 중 글자와 점이 찌그러졌습니다.
  - 측정에서 CPU 4배 감속 조건의 21만 점 Scatter 차트가 느린 것을 확인했습니다. 출시 전부터 드문 대용량 사례에 맞추기보다 일반적인 화면의 선명도를 우선하기로 했습니다.
  - 캔버스 확대와 지연 처리를 제거하고 requestAnimationFrame으로 프레임당 한 번 실제 크기에 맞춰 그리도록 바꿨습니다.

MaxGauge 차세대 (계속)

- **러너 증설 전에 실행 비용을 분석해 CI 시간 단축**
  - CI가 느려졌지만 큐 대기 중앙값은 1.6초여서 실행 단계의 비용부터 살펴봤습니다.
  - Storybook을 미리 빌드한 파일로 제공하고, DOM이 필요 없는 테스트는 Node 환경에서 실행하도록 바꿨습니다. 중복 분석과 리포트 생성도 줄였습니다.
  - 당시 파이프라인 시간 중앙값은 198초에서 94초로 줄었습니다. 변경 전 20회와 후 4회를 비교한 기록이며, 재시도와 캐시 재생성은 제외했습니다.

Task Analyzer | 작업 맥락 검색 도구 실험 프로젝트

`Python / TypeScript / MCP`

- 개발에 필요한 요구사항과 관련 코드, 과거 작업 이력을 여러 AI 도구에서 찾을 수 있도록 데이터 관계와 검색 기능을 별도 서버로 분리했습니다.
- MCP와 스킬에서 같은 검색 기능을 쓰도록 규격을 정하고 사내 플러그인 마켓에 구성했습니다.
- 회의록에 관련 태스크 링크를 남기거나 개발 중 작업 이력을 찾는 데 사용하고 있습니다.

GitLab 슬래시 커맨드 | MR 스레드에서 AI와 협업 실험 프로젝트

`TypeScript / LangGraph`

- 작업 맥락을 개인 PC로 옮기지 않고 MR 댓글에서 AI를 호출해, 답변과 수정 결과를 같은 스레드에서 함께 검토하도록 만들었습니다.
- 서로 다른 논의가 섞이지 않도록 대화 이력은 스레드별로 분리하고, 작업 결과 요약은 MR 안에서 공유하도록 했습니다.

학력

동의대학교 정보통신공학과 / 응용 소프트웨어 공학 부전공 | 2017.03 - 2024.03

최근 글 전체 글 최신 5개 / 2026.10.25 확인

자본주의 1부 - 돈은 빚이다
2026.09.20

지금 테스트 코드가 더더욱 중요한 이유
2026.09.06

스틱 후기
2026.08.29

Tanstack Table V9 마이그레이션 회고
2026.08.26

개발자로 살아남기 - 성장에 대한 고민
2026.08.09